Abstract Data Type

List

Thought Exercise (Check out the notes on Asymptotic Analysis)

- Prove/disprove the following
  - 23 is O(log n)
  - 5 log n is O (1)
  - 5n - 7 is O(n)



Preliminaries

- Navigating file system
- Browsing phonebook
- Logging in to facebook
- Google searches
- Waze or Google Maps



Abstract Data Type

- A mathematical model composed of a set of values and operations that operate (access or manipulate) on the set of values



List

- Container
  - Items
  - Operations
    - add/insert items
      - insert
      - push_front
      - push_back



List

- Operations
  - remove/erase items
    - erase
    - pop_front
    - pop_back



List

- Operations
  - accessing items
    - front
    - back
    - iterator
      - begin
      - end



List

- Abstract
  - Implementation is hidden
- Downside
  - When accessing arbitrary positions is paramount, the use of the list is not the best choice



array container

- Fixed-size sequence container
- Keeps only the array as data
- Operations
  - access
  - insert items
  - remove items



Why the primitive array is problematic

- Size is not known
  - Bounds (subscript) are not checked
- Heap memory allocation
- Manual item insertion
- Passed by reference



Review on Classes

- Blueprint of an object
  - defines state
    - characteristics
    - data members
  - defines behaviour
    - operations/methods
    - communication



Sample

- bank account
  - data members
    - account name
    - account number
    - type
    - balance



Sample

- bank account
  - operations
    - balance inquiry
    - deposit
    - withdraw
    - close



Array List

- Fixed-size sequence container
- Keeps only the array as data and the size
- Operations
  - access
  - insert items
  - remove items



Show me the Money

- bank account
  - data members
    - account name
    - account number
    - account type
    - balance



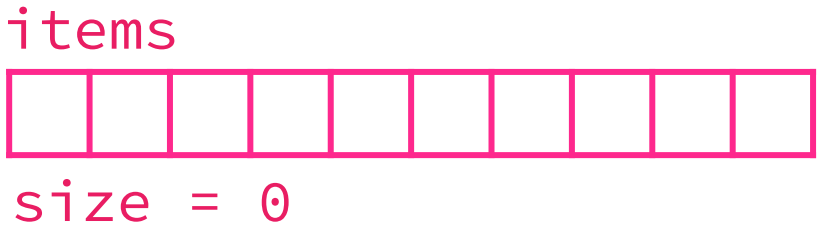
Show me the Money

- bank account
  - operations
    - open an account
    - deposit into an account
    - withdraw from an account
    - close an account



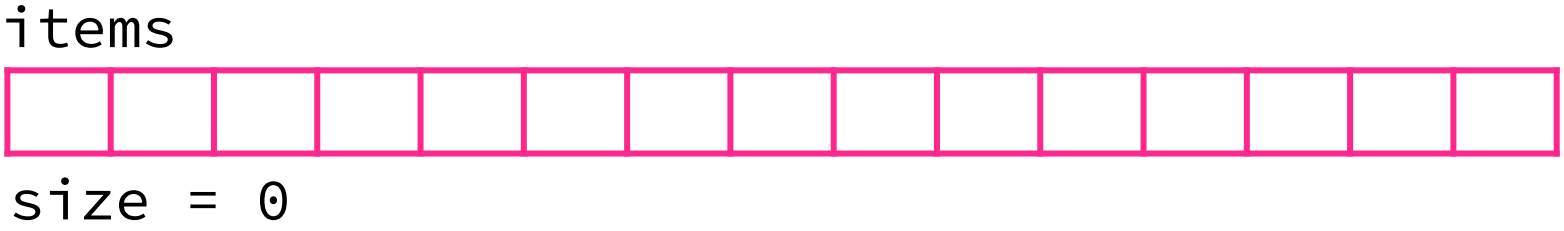
Basic ArrayList of ints

```
class ArrayList{
   private:
        int items[MAX];
        int size;
   public:
        ArrayList();
   };
void ArrayList::ArrayList(){
   l->size = 0;
}
```



Show me the Money

- opening an account in a bank is like adding an item into the collection (ArrayList)
  - append or push_back



Show me the Money

```
bool ArrayList::push_back(int x){
    if(size == MAX)
        return false;
    else{
        items[size++] = x;
        return true;
    }
}
```

items



size = 0



Show me the Money

```
___
```

- ```
  bool ArrayList::push_back(int x){
      if(size == MAX)
          return false;
      else{
          items[size++] = x;
          return true;
      }
  }
  ```

items



size = 1
  ```

Show me the Money

```
---
```

```
bool ArrayList::push_back(int x){
    if(size == MAX)
        return false;
    else{
        items[size++] = x;
        return true;
    }
}
```

items



size = 2

Show me the Money

```
___
❑  bool ArrayList::push_back(int x){
       if(size == MAX)
           return false;
       else{
           items[size++] = x;
           return true;
       }
   }
```

items



size = 3

Show me the Money

- ```
  bool ArrayList::push_back(int x){
      if(size == MAX)
          return false;
      else{
          items[size++] = x;
          return true;
      }
  }
  ```

items



size = 4
  ```

Show me the Money

```
bool ArrayList::push_back(int x){
    if(size == MAX)
        return false;
    else{
        items[size++] = x;
        return true;
    }
}
```

items



size = 5

Show me the Money

```
bool ArrayList::push_back(int x){
    if(size == MAX)
        return false;
    else{
        items[size++] = x;
        return true;
    }
}
```

items



size = 6

Show me the Money

```
___
❑  bool ArrayList::push_back(int x){
       if(size == MAX)
           return false;
       else{
           items[size++] = x;
           return true;
       }
   }
```

items



size = 7

Show me the Money

```
___
```

- ```
  bool ArrayList::push_back(int x){
      if(size == MAX)
          return false;
      else{
          items[size++] = x;
          return true;
      }
  }
  ```

items



size = 8
  ```

Show me the Money

```
---
```

```
bool ArrayList::push_back(int x){
    if(size == MAX)
        return false;
    else{
        items[size++] = x;
        return true;
    }
}
```

items



size = 9

Show me the Money

```
bool ArrayList::push_back(int x){
    if(size == MAX)
        return false;
    else{
        items[size++] = x;
        return true;
    }
}
```

items



size = 10

Show me the Money

```
___
❑  bool ArrayList::push_back(int x){
       if(size == MAX)
           return false;
       else{
           items[size++] = x;
           return true;
       }
   }
```

items



size = 10

Show me the Money - pos 1

```
___
❑  bool ArrayList::push_back(int x){
        if(size == MAX)
            return false;
        else{
            items[size++] = x;
            return true;
        }
    }
```

items



size = 10

Show me the Money - pos 1

```
___
❑  bool ArrayList::push_back(int x){
        if(size == MAX)
            return false;
        else{
            items[size++] = x;
            return true;
        }
    }
```

items



size = 10

Show me the Money - Closing an account is like removing an item from the collection (say the account is in position 1)

```
bool ArrayList::erase(int pos){
    if(pos < 0 || pos >= size)
        return false;
    else{
        for(int i=pos; i < size-1; i++)
            items[i] = items[i+1];
        size--;
        return true;
    }
}
```

items



size = 10

Show me the Money - Closing an account is like removing an item from the collection (say the account is in position 1)

```
bool ArrayList::erase(int pos){
    if(pos < 0 || pos >= size)
        return false;
    else{
        for(int i=pos; i < size-1; i++)
            items[i] = items[i+1];
        size--;
        return true;
    }
}
```

items



size = 10

Show me the Money - Closing an account is like removing an item from the collection (say the account is in position 1)

```
bool ArrayList::erase(int pos){
    if(pos < 0 || pos >= size)
        return false;
    else{
        for(int i=pos; i < size-1; i++)
            items[i] = items[i+1];
        size--;
        return true;
    }
}
```

items



size = 10

Show me the Money - Closing an account is like removing an item from the collection (say the account is in position 1)

```
bool ArrayList::erase(int pos){
    if(pos < 0 || pos >= size)
        return false;
    else{
        for(int i=pos; i < size-1; i++)
            items[i] = items[i+1];
        size--;
        return true;
    }
}
```

items



size = 10

Show me the Money - Closing an account is like removing an item from the collection (say the account is in position 1)

```
bool ArrayList::erase(int pos){
    if(pos < 0 || pos >= size)
        return false;
    else{
        for(int i=pos; i < size-1; i++)
            items[i] = items[i+1];
        size--;
        return true;
    }
}
```

items



size = 10

Show me the Money - Closing an account is like removing an item from the collection (say the account is in position 1)

```
bool ArrayList::erase(int pos){
    if(pos < 0 || pos >= size)
        return false;
    else{
        for(int i=pos; i < size-1; i++)
            items[i] = items[i+1];
        size--;
        return true;
    }
}
```

items



size = 10

Show me the Money - Closing an account is like removing an item from the collection (say the account is in position 1)

```
bool ArrayList::erase(int pos){
    if(pos < 0 || pos >= size)
        return false;
    else{
        for(int i=pos; i < size-1; i++)
            items[i] = items[i+1];
        size--;
        return true;
    }
}
```

items



size = 10

Show me the Money - Closing an account is like removing an item from the collection (say the account is in position 1)

```
bool ArrayList::erase(int pos){
    if(pos < 0 || pos >= size)
        return false;
    else{
        for(int i=pos; i < size-1; i++)
            items[i] = items[i+1];
        size--;
        return true;
    }
}
```

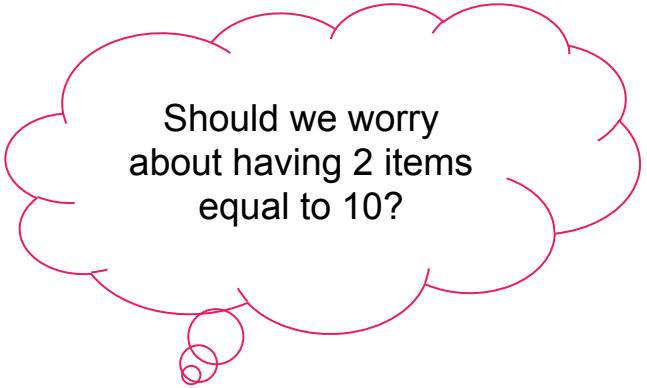
items



size = 9

Show me the Money - Closing an account is like removing an item from the collection (say the account is in position 1)

```
bool ArrayList::erase(int pos){
    if(pos < 0 || pos >= size)
        return false;
    else{
        for(int i=pos; i < size-1; i++)
            items[i] = items[i+1];
        size--;
        return true;
    }
}
```



items



size = 9

List general insert (based on position)

---

items



size = 9

List general insert (based on position) - 20 at pos 9

---

items



size = 9

List general insert (based on position) - 20 at pos 9

---

items



size = 9

List general insert (based on position) - 15 at pos 2

---

items



size = 10

List general insert (based on position) - 15 at pos 2

---

items



size = 10



List general insert (based on position) - 15 at pos 2

---

items



size = 10

List general insert (based on position) - 15 at pos 2

---

items



size = 9

List general insert (based on position) - 15 at pos 2

---

items



size = 9



List general insert (based on position) - 15 at pos 2

---

items



size = 9



List general insert (based on position) - 15 at pos 2

---

items



size = 9

List general insert (based on position) - 15 at pos 2

---

items



size = 9

List general insert (based on position) - 15 at pos 2

---

items



size = 9

List general insert (based on position) - 15 at pos 2

---

items



size = 9

List general insert (based on position) - 15 at pos 2

---

items



size = 10

List general insert (based on position) - 15 at pos 2

---

items



size = 10